

WiDS 5.0: Stock Prediction using Reinforcement Learning

Final Project Report

Mentee:

Vithulakash

Mentors:

Vaidehi Agrawal

Abhinav Badegaonkar

January 2026

Introduction

The objective of this project is to implement a reinforcement learning-based stock trading agent specifically for the Indian market using NIFTY 50 data. This report details the transition from basic finance and RL concepts to a complex multi-asset environment based on the “Stock prediction ensemble strategy” paper. The implementation focuses on balancing exploration and exploitation to maximize portfolio returns within a custom-built Markov Decision Process framework.

Financial Graphs and Indicators

1.1 Candle stick charts

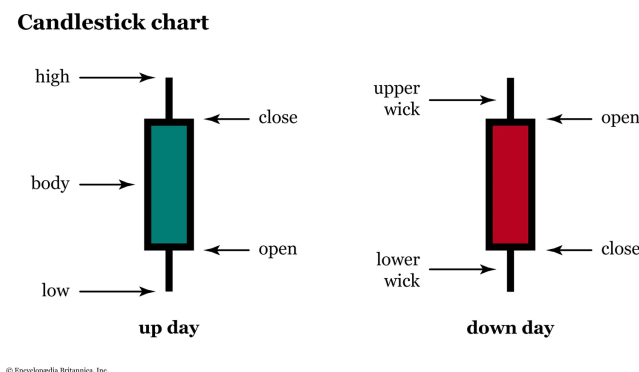


Figure 1.1: Candle sticks

- Candle stick charts are very commonly used to analyze and interpret the market as it is a very easy to read depiction and it carries all necessary data
- Various candle sticks patterns can be analyzed together to find the current trend in the market.

1.2 Technical Indicators

1. **Moving Average Convergence Divergence (MACD):** A trend-following momentum indicator calculated by subtracting the 26-period EMA from the 12-period EMA. It helps identify changes in the strength, direction, momentum, and duration of a trend.
2. **Relative Strength Index (RSI):** A momentum oscillator that measures the speed and change of price movements on a scale of 0 to 100. It is primarily used to identify overbought or oversold conditions to signal potential reversals.
3. **Commodity Channel Index (CCI):** An oscillator that compares the current price to its average price over a specific time period. It is used to spot new trends or extreme conditions when prices deviate significantly from their historical average.
4. **Average Directional Index (ADX):** A trend strength indicator that quantifies the intensity of a trend regardless of its direction. High values indicate a strong trending market, while low values suggest a weak or sideways market.

Literature Review & Theory

2.1 Markov Decision Process (MDP)

Trading is modeled as an MDP where an agent interacts with the market environment over discrete time steps. The transition from state S_t to S_{t+1} is governed by the agent's actions a_t , which are determined by the policy $\pi(a|s)$. Understanding these dynamics forms the foundation of modern reinforcement learning.

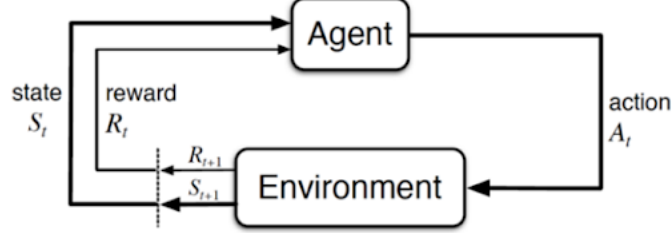


Figure 2.2: MDP Model

2.2 Reinforcement Learning Fundamentals

Reinforcement Learning (RL) frames the problem of stock trading as an agent learning to make decisions by interacting with an environment to maximize cumulative reward. This section outlines the core mathematical components used in this project.

2.2.1 Policies (π)

The behavior of an agent is defined by its **policy**, denoted as π , which maps a given state s to an action a .

- **Stochastic Policy:** $\pi(a|s) = \mathbb{P}[A_t = a|S_t = s]$. This allows the agent to explore by selecting actions based on probability distributions.
- **Deterministic Policy:** $a = \pi(s)$. This maps a state directly to a specific action, which is essential for algorithms operating in continuous action spaces like DDPG or PPO.

2.2.2 Value Functions

To evaluate the quality of a policy, we utilize value functions which estimate expected future rewards.

The State-Value Function (V^π)

The state-value function $V^\pi(s)$ estimates the expected cumulative reward starting from state s and following policy π thereafter. It answers the question: "How good is it to be in this state?"

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right] \quad (2.1)$$

where $\gamma \in [0, 1]$ is the discount factor that weighs the importance of immediate versus future rewards.

The Action-Value Function (Q^π)

The action-value function $Q^\pi(s, a)$, often referred to as the Q-function, estimates the value of taking action a in state s , and subsequently following policy π :

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right] \quad (2.2)$$

In the context of stock trading, $Q^\pi(s, a)$ essentially represents the expected long-term profit of a specific buy/sell action given current market conditions.

2.3 Learning Methodologies

An agent calculates these value functions using experience generated from the environment. Two primary methodologies for this are Monte Carlo (MC) methods and Temporal Difference (TD) learning.

2.3.1 Monte Carlo (MC) Methods

Monte Carlo methods update value estimates by averaging returns observed after the completion of an entire episode. The value of a state is updated only after the terminal state is reached. The update rule for $V(S_t)$ is:

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)] \quad (2.3)$$

where G_t is the actual total return accumulated from time t to the end of the episode, and α is the step size.

- **Advantage:** MC estimates are unbiased because they rely on actual observed returns.
- **Disadvantage:** High variance, as returns depend on many random actions and transitions throughout the episode. It strictly requires episodic tasks.

2.3.2 Temporal Difference (TD) Learning

TD learning combines ideas from Monte Carlo and Dynamic Programming. Unlike MC, TD methods update estimates based on other learned estimates, a process known as *bootstrapping*. The agent updates its value function after every time step rather than waiting for the episode to end. The TD(0) update rule is:

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)] \quad (2.4)$$

Here, the term $R_{t+1} + \gamma V(S_{t+1})$ acts as the target.

- **Advantage:** Lower variance than MC and capability to learn online during the episode.
- **Disadvantage:** The estimates are biased initially because they rely on the current (potentially inaccurate) estimate of $V(S_{t+1})$.

2.3.3 Bias-Variance Trade-off

The choice between MC and TD represents a fundamental trade-off in RL. MC provides an unbiased estimate of the return but suffers from high variance due to the stochastic nature of long trajectories. TD targets are biased because they bootstrap but have lower variance because they depend only on the immediate reward and the next state. Algorithms like TD(λ) offers a middle ground, blending MC and TD updates to balance this trade-off effectively.

2.4 Deep Q-Networks (DQN) and Preliminary Work

While tabular TD learning is effective for small state spaces, it becomes intractable for complex environments with high-dimensional states. To address this, Deep Q-Networks (DQN) utilize neural networks as function approximators to estimate the Q-values, $Q(s, a; \theta) \approx Q^*(s, a)$.

2.4.1 DQN Architecture and Loss

In a DQN, the network parameters θ are optimized by minimizing the mean squared error between the predicted Q-values and the target Q-values derived from the Bellman equation:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim D} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (2.5)$$

Here, θ^- represents the parameters of a separate *target network* that is frozen for a fixed number of steps to stabilize training, and D represents the Experience Replay buffer.

2.4.2 Midterm Implementation: Lunar Lander

As a precursor to the stock prediction agent, a DQN was implemented to solve the *Lunar Lander* environment. This served as a validation of value-based Deep RL methods in discrete action spaces.

Network Structure

The agent utilized a Feedforward Neural Network with the following architecture:

- **Input Layer:** Corresponding to the state vector size.
- **Hidden Layers:** Two fully connected layers with 128 neurons each.
- **Activation:** Rectified Linear Unit (ReLU) applied after each hidden layer.
- **Output Layer:** Linear output corresponding to the Q-values for each discrete action.

Experience Replay

To mitigate the correlation between consecutive samples, an Experience Replay mechanism was implemented using a First-In-First-Out (FIFO) deque. Transitions (s_t, a_t, r_t, s_{t+1}) were stored in this buffer, and training involved random sampling of mini-batches. This technique breaks the temporal correlations in the observation sequence, significantly improving the stability of the neural network updates.

Training Strategy

The agent employed an ϵ -greedy strategy for exploration, initializing with 100% exploration ($\epsilon = 1.0$) and linearly decaying the probability of random actions over time. The optimization process involved backpropagating gradients through the network to update the policy toward the target policy at every step, provided the replay buffer contained sufficient samples.

Experimental Observations: DQN vs. DDQN

Double DQN (DDQN) was also explored to reduce the overestimation bias inherent in standard Q-learning. However, empirical testing on the Lunar Lander environment revealed that the DDQN implementation yielded noisy and unstable training curves compared to the standard DQN. Consequently, the standard DQN with carefully tuned hyperparameters (learning rate, batch size, and buffer size) was selected as the final model for the midterm submission.

2.5 Policy Gradient Methods

While value-based methods like Q-Learning attempt to learn a value function and deduce a policy from it (e.g., by acting greedily), **Policy Gradient** methods take a different approach. They directly parameterize the policy $\pi_\theta(a|s)$ and optimize the parameters θ to maximize the expected return.

This approach is particularly advantageous for stock trading for two reasons:

1. **Continuous Action Spaces:** Trading often involves deciding how much to buy or sell, which is a continuous value. Value-based methods struggle here, whereas policy gradients can output a continuous distribution (e.g., a Gaussian).
2. **Stochastic Policies:** Markets are partially observable and noisy. Policy gradients naturally learn stochastic policies, preventing the agent from getting stuck in deterministic loops.

2.5.1 The Objective Function

The goal is to maximize the expected cumulative reward $J(\theta)$. For a trajectory $\tau = (s_0, a_0, r_1, s_1, \dots)$, the objective is defined as:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] \quad (2.6)$$

where $R(\tau)$ is the total return of the trajectory. We aim to update the policy parameters θ via gradient ascent:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta J(\theta_t) \quad (2.7)$$

2.5.2 The Policy Gradient Theorem

Computing the gradient $\nabla_\theta J(\theta)$ is non-trivial because the environment's dynamics are unknown. However, the **Policy Gradient Theorem** provides a solution by reformulating the gradient as an expectation that can be estimated from samples:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \ln \pi_\theta(a_t|s_t) Q^{\pi_\theta}(s_t, a_t)] \quad (2.8)$$

This theorem relies on the "log-derivative trick," which allows us to calculate the direction of steepest ascent without knowing the state transition probabilities. Intuitively, this update rule increases the probability of actions that yield high Q -values (good returns) and decreases the probability of actions with low Q -values.

2.5.3 Actor-Critic Architectures

Pure policy gradient methods (like REINFORCE) suffer from high variance because they rely on the full Monte Carlo return G_t to estimate $Q(s, t)$. To mitigate this, we employ **Actor-Critic** methods, which are the foundation of the algorithms used in this project (such as PPO).

In this framework, the agent is composed of two neural networks:

- **The Actor (π_θ):** Responsible for selecting actions. It is updated using the policy gradient.
- **The Critic (V_ϕ or Q_ϕ):** Responsible for evaluating the actions taken by the Actor by estimating the value function.

By using the Critic's estimate (a bootstrap) instead of the raw Monte Carlo return, we significantly reduce the variance of the gradient updates, stabilizing the learning process in volatile environments like stock markets.

Final Project

3.1 Data Collection and Preprocessing

3.1.1 Data Source and Cleaning

Daily NIFTY 50 OHLCV (Open, High, Low, Close, Volume) data spanning from 2010 to 2019 was utilized. To ensure data integrity, all stocks were aligned by date to ensure synchronization, and missing values were handled by removing the day across all stocks.

3.1.2 Feature Engineering

To capture market momentum and trends, standard technical indicators were computed for each stock and added to the state space:

- **MACD:** Captures trend direction and strength.
- **RSI:** Identifies overbought or oversold conditions.
- **CCI & ADX:** Measure trend strength and cyclical deviations.

3.1.3 Normalization and Scaling

Normalization is a critical step in Reinforcement Learning, particularly when dealing with financial data where raw values can vary wildly in magnitude.

The Need for Normalization

The state space consists of heterogeneous features: stock prices (ranging from 100 to 10,000+), account balances (in lakhs), and technical indicators (like RSI bounded between 0-100, daily return usually single digit decimal). If fed raw data into the neural network, the large magnitude features (like Price or Balance) would produce massive gradients that overshadow the smaller but highly informative features (like MACD). Normalization brings all inputs to a similar scale, ensuring the optimizer treats all features with equal importance.

Rolling Window Normalization

Standard normalization techniques (like Standard Scaler) often introduce **look-ahead bias** by using future global minimums or maximums to scale current data. To prevent this "data leakage," a rolling window normalization was applied.

- **Method:** At any time t , the data is scaled using mean and variance statistics derived only from a look-back window $[t - w, t]$.
- **Benefit:** This ensures the agent's decision at time t is based strictly on information available up to that moment, simulating a realistic live-trading environment.

Scaled State Observations

Finally, the environment was designed to return scaled portfolio values for the observation state. Instead of returning the raw cash balance (e.g., ₹1,000,000), the state returns the *portfolio weight* or a scaled factor relative to the initial capital. This keeps the neural network's input layer activations within a stable range, preventing saturation of activation functions (like Tanh) and facilitating faster, more stable convergence during training.

3.2 Environment Formulation

3.2.1 State Space (S_t)

The state vector is mathematically formulated as:

$$S_t = [b_t, P_t, h_t, M_t, R_t, C_t, X_t]$$

This vector encapsulates the current cash balance (b_t), stock prices (P_t), and holdings (h_t), alongside technical market indicators. Two models were trained one was trained with raw data convention, the state space returning actual value of cash balance and stock prices, the other was trained on state space returning scaled and normalized values

3.2.2 Action Space

The action space $a_t \in [-1, 1]^D$ represents the decision to buy, sell, or hold for each stock in the portfolio. These continuous values are mapped to actual share changes based on a maximum share threshold h_{max} :

$$\Delta h_t^{(d)} = [a_t^{(d)} \cdot h_{max}]$$

3.2.3 Reward Function

The reward r_t is designed to reflect the net change in portfolio value, explicitly accounting for a 0.1% transaction cost c_t per trade:

$$r_t = (b_{t+1} + p_{t+1}^\top h_{t+1}) - (b_t + p_t^\top h_t) - c_t$$

3.3 Implementation Details

The custom environment was built to be compatible with standard RL libraries using Python and PyTorch. The selection of the final algorithm was the result of a comparative study involving multiple actor-critic architectures.

3.3.1 Algorithm Selection: A2C vs. DDPG vs. PPO

Before finalizing the model, three distinct reinforcement learning algorithms were implemented and evaluated on a subset of the data:

1. **A2C (Advantage Actor-Critic):** A synchronous, deterministic variant of Asynchronous Advantage Actor-Critic (A3C). While faster to train, it struggled to converge stably in the highly volatile market environment, often resulting in erratic portfolio performance.
2. **DDPG (Deep Deterministic Policy Gradient):** Designed specifically for continuous action spaces, DDPG was a strong candidate. However, it proved highly sensitive to hyperparameter tuning and suffered from instability due to the non-stationarity of financial time series data.
3. **PPO (Proximal Policy Optimization):** PPO emerged as the superior choice. Its clipped objective function prevents large, destructive policy updates, providing the stability required for financial data. It offered the best balance of sample efficiency, ease of tuning, and consistent performance across varying market conditions.

3.3.2 Hyperparameter Configuration

Following the selection of PPO, both the algorithm and environment parameters were tuned to optimize performance:

- **Learning Rate** ($\alpha = 0.0003$): A standard value for the Adam optimizer was selected. During preliminary runs, higher rates ($> 10^{-3}$) caused the policy to diverge due to the high variance in stock returns, while lower rates ($< 10^{-4}$) resulted in prohibitively slow convergence.
- **Horizon Length** ($n_steps = 2048$): This parameter defines the number of steps the agent interacts with the environment before a policy update occurs. A relatively long horizon of 2048 steps was chosen to ensure the agent captures longer-term market trends and collects a diverse batch of transitions before calculating the advantage estimates.
- **Optimization Epochs** ($n_epochs = 15$): This determines how many times the optimizer passes over the collected buffer data per update cycle. Setting this to 15 allowed the agent to squeeze more "learning" out of each batch of experience without overfitting to the specific noise of that batch.
- **Batch Size** (64): Used for the inner loop of the PPO update. A smaller batch size of 64 provided a regularizing effect, introducing enough noise into the gradient updates to prevent the agent from getting stuck in sharp local optima.
- **Maximum Share Limit** (h_{max}): This environment constraint dictates the maximum number of shares the agent can buy or sell in a single action. It was tuned from 100 up to 5000.
 - *Low* (100): Initially, the agent's actions lacked impact; even correct "buy" signals resulted in negligible portfolio growth due to limited volume.
 - *High* (> 1000): Extremely high limits proved disastrous. Single failed trades caused massive drawdowns, significantly depleting the cash balance.
 - *Optimal* (500 – 1000): This range struck the correct balance, providing the agent enough buying power to capitalize on trends without exposing the portfolio to ruin risk on a single bad trade.

3.3.3 Tuning Methodology and Convergence

The primary variable for tuning was the **Total Timesteps**, which dictates the total duration of the training phase.

- **Range Tested:** The agent was evaluated at intervals of 100k, 300k, and 500k timesteps.
- **Observations:** At lower timestep counts (100k), the agent often exhibited "underfitting," failing to react to sudden market downturns.
- **Final Selection:** The training was extended to **500,000 timesteps**. This duration was critical to ensure the agent experienced a sufficient variety of market conditions, allowing the value function to converge and the policy to stabilize.

3.3.4 Training and Validation Strategy

The dataset was split chronologically, with the first 80% utilized for training and the remaining 20% reserved for out-of-sample testing. To ensure the sanity of the learning process during the training phase, key metrics were monitored numerically: ****Explained Variance**** was tracked to verify the Critic's predictive power (targeting values near 1.0), while ****Policy and Value Losses**** were observed to ensure they stabilized over time without diverging.

3.4 Results and Evaluation

3.4.1 Performance Metrics

The primary evaluation metrics chosen for this project were Cumulative Returns and the Sharpe Ratio. While cumulative returns indicate total profitability, they fail to account for the exposure to volatility required to achieve those gains.

The Sharpe Ratio

To provide a more nuanced evaluation, the **Sharpe Ratio** was utilized to measure risk-adjusted performance. It quantifies the excess return generated per unit of deviation (risk). Mathematically, it is defined as:

$$S = \frac{\mathbb{E}[R_p - R_f]}{\sigma_p} \quad (3.9)$$

Where:

- R_p is the return of the agent's portfolio.
- R_f is the risk-free rate (typically the return of a safe asset like government bonds, set to 0 for this relative comparison).
- σ_p is the standard deviation of the portfolio's excess return.

Significance: A higher Sharpe ratio indicates that the agent is not merely "getting lucky" by taking reckless gambles on volatile stocks. Instead, it demonstrates that the agent has learned a robust policy that maximizes yield while minimizing exposure to unnecessary market turbulence.

3.4.2 Portfolio Visualizations

Cumulative returns were plotted over the evaluation period to visualize the agent's trajectory and stability.

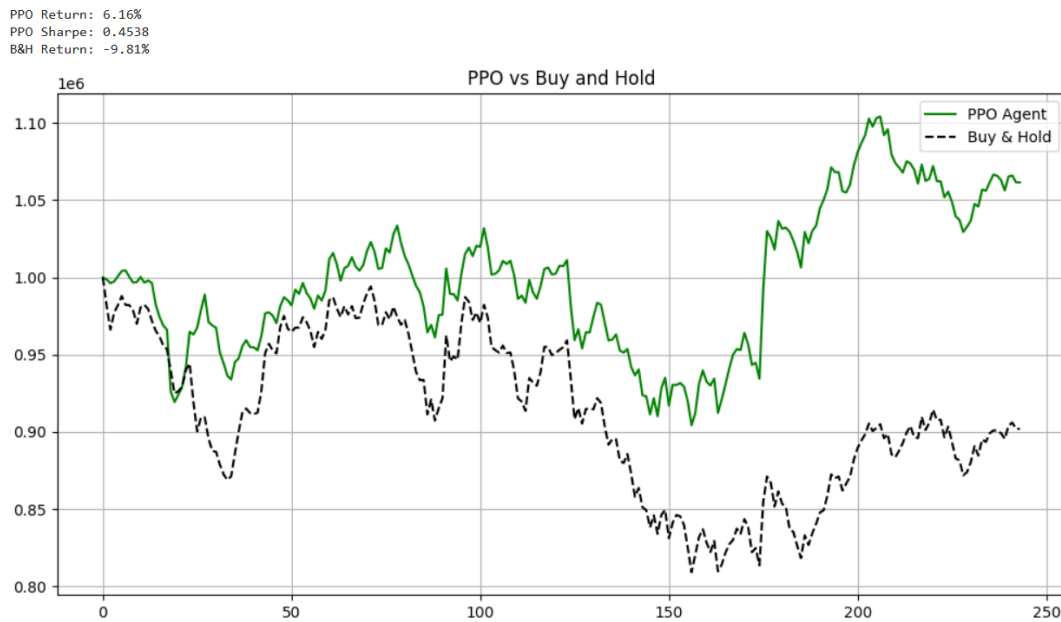


Figure 3.3: Model trained on normalized data

Metric	PPO Agent	Buy & Hold
Total Return	11.88%	-9.81%
Sharpe Ratio	0.8586	-0.5193

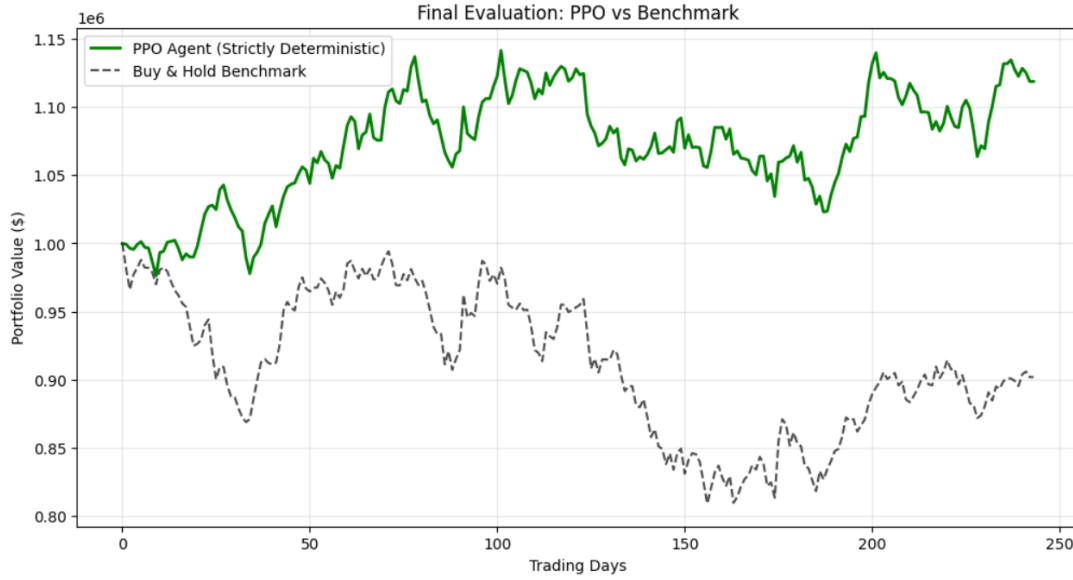


Figure 3.4: Model trained on raw data

3.4.3 Reproducibility and Stochasticity

Achieving perfect reproducibility here is challenging. While random seeds were fixed for Python, NumPy, and PyTorch to control initialization and exploration noise, minor variations in training results persisted. This is primarily attributed to non-deterministic CUDA operations on the GPU and the asynchronous nature of parallelized environments, which can slightly alter the order of experience collection. As seeding other parameters didn't help none were seeded in the submission.

3.5 Discussion: The Effect of Normalization

An unexpected and significant finding during the evaluation phase was the comparative performance of agents trained on raw versus normalized state data. Contrary to standard Deep Learning conventions, the agent trained on raw feature inputs outperformed the normalized agent, achieving a Sharpe ratio of 0.85 compared to 0.45.

3.5.1 Visual Analysis of Trajectories

A critical insight is visible when comparing the equity curves of both models against the Buy & Hold benchmark:

- **Normalized Model:** The trajectory of the normalized agent closely mirrors the market benchmark. It exhibits high correlation with the NIFTY 50 index, suggesting that the normalization process caused the agent to learn a conservative, "index-tracking" policy rather than an active trading strategy. The normalization made the model value everything equally so, it closely understood the trend of the market and learned to mimic it with minimal risk and somewhat better returns.
- **Raw Model:** In contrast, the raw model's trajectory "decouples" from the benchmark. It demonstrates the ability to avoid drawdowns during market dips and accelerates upward during trends, effectively generating excess returns rather than just market exposure.

3.5.2 Hypothesis: Feature Dominance vs. Noise Overfitting

This performance gap is likely attributed to how the network prioritizes information:

- **Raw Data Regime:** The NIFTY 50 index values (ranging from 5,000 to 12,000) possess a significantly larger magnitude than technical indicators like RSI (0–100). This magnitude forces the network to prioritize the **Price** feature. Since the market was in a strong trend, this "forced focus" allowed the agent to latch onto the primary price signal and ignore contradictory noise from secondary indicators.
- **Normalized Regime:** By scaling all features to $[-1, 1]$, the normalized model treated noisy indicators (like a fluctuating ADX) with the same importance as the price trend. The agent likely struggled to separate the signal from the noise, resulting in a hesitant policy that simply mimicked the market average to minimize the penalty of wrong actions.

3.5.3 Algorithm Robustness

Typically, raw data leads to exploding gradients. However, the use of **Proximal Policy Optimization (PPO)** mitigated this risk. PPO's objective function clipping (set to $\epsilon = 0.2$) effectively prevented the destructive policy updates that would usually occur with unscaled high-magnitude inputs, allowing the agent to benefit from the unscaled price signal without diverging.

3.6 Conclusion and Learnings

By the end of this project we successfully implemented a Reinforcement Learning-based trading agent for the Indian stock market (NIFTY 50), validating the potential of autonomous systems in financial decision-making. By leveraging the Proximal Policy Optimization (PPO) algorithm, the agent demonstrated the ability to navigate complex market dynamics, outperforming the buy-and-hold benchmark.

3.6.1 Key Learnings

The transition from theoretical reinforcement learning to practical application revealed several critical insights that contradict standard Deep Learning heuristics:

1. **Context-Aware Preprocessing:** Standard normalization techniques, while mathematically sound for image or text data, can be detrimental in financial time series. We observed that the agent trained on **raw data** significantly outperformed the normalized agent (Sharpe 0.85 vs. 0.45). The normalized agent effectively learned to "track the index" to minimize risk, whereas the raw agent utilized the magnitude of price data to identify and exploit aggressive trends.
2. **Stability over Complexity:** While complex algorithms like DDPG offer continuous control, they proved too unstable for the non-stationary noise of the stock market. PPO's "clipped" objective function was the decisive factor in achieving convergence, proving that algorithmic stability is often more valuable than theoretical expressiveness in financial domains.
3. **The Risk-Reward Reality:** High cumulative returns are often misleading. The extensive tuning of the h_{max} (share limit) parameter highlighted the delicate balance between buying power and ruin risk. To be profitable we must also learn to keep risk under control.